

HASHDIT

HashDit

Smart Contract Security

ListaDao PositionMigrator Batch Force Migration

Security Audit Report

ListaDao

13 Oct 2026 – 15 Oct 2026

Disclaimer

This report is provided on an "as is" basis and does not constitute investment advice, a recommendation, or an endorsement of the audited project. HashDit makes no representations or warranties regarding the safety, functionality, or correctness of the smart contracts beyond the scope defined herein.

The audit was performed based on the source code provided at the time of the engagement. Any subsequent modifications to the codebase may invalidate the findings of this report. HashDit shall not be held liable for any losses or damages resulting from the use, deployment, or interaction with the audited contracts.

The scope of this audit is limited to the smart contracts listed in this report. It does not cover off-chain components, front-end interfaces, or third-party integrations unless explicitly stated. Users and developers should conduct their own due diligence before interacting with or deploying the audited contracts.

Audit Overview

Audit Period	13 Oct 2026 - 15 Oct 2026
Overall Risk	Medium
Project Name	ListaDao
Github	https://github.com/lista-dao/moolah/pull/254
Commit	feat/pm-batch-force-migrate

Smart Contract List

No.	Contract Name	Verdict	Details
1	PositionMigrator.sol	Medium	[M01] [M02] [L01] [I01] [I02] [I03] [I04]

Findings

[M01] Planned slisBNB market reallocation leaves only a ~40k buffer and near 100% utilization	
Contract	PositionMigrator.sol
Severity	Medium
Description	**Description** Currently, the slisBNB target market has ~709,004 lisUSD of borrowable liquidity. The plan reallocates ~1.11M into it to cover ~1.78M of combined BNB and slisBNB debt, which yields roughly 1.82M of capacity against 1.78M of demand, a buffer of about 40k, or near 2%. **Impact** Interest keeps accruing on the market's 18.3M of existing borrow between planning and execution, and the exact migrated debt is read at run time, so a small increase can erase the 40k buffer and trigger INSUFFICIENT_LIQUIDITY, reverting the batch. Even on success the migration drives the market to about 99.8% utilization. At that utilization existing lisUSD suppliers cannot withdraw, because Moolah.withdraw requires totalBorrowAssets <= totalSupplyAssets, and the variable borrow rate spikes for the migrated users.
Recommendation	Consider: • Providing a larger safety buffer in the slisBNB reallocation rather than matching demand almost exactly, and recompute aggregate debt immediately before execution so accrued interest is included. • Splitting the slisBNB migration across several smaller batches so a transient shortfall fails only one small batch instead of the entire set. Communicate the expected utilization spike and its effect on supplier withdrawals and borrow rates.
Status	[Open/Fixed/Acknowledged]

[M02] BTCB target market reallocation of 94k is short of the 795,191 lisUSD BTCB CDP debt by 104,774	
Contract	PositionMigrator.sol
Severity	Medium
Description	**Description** The BTCB target market (MARKET_BTCB) currently holds 1,070,821 lisUSD of supply against 474,404 lisUSD of borrow, leaving 596,417 lisUSD of borrowable liquidity. The operational plan reallocates +94,000 into this market, giving 690,417 lisUSD of capacity. However, the total BTCB debt on the CDP, read from the Vat as Art * rate / 1e27, is 795,191 lisUSD.

	Impact Since only CDP positions can migrate and the 4 ilk debt totals sum to 2,577,491 lisUSD, matching the ~2.58M whitelisted figure, essentially all 795,191 of BTCB debt is in scope. During a forced migration the contract re-borrows each owner's debt from the target market, and Moolah.borrow reverts with INSUFFICIENT_LIQUIDITY once cumulative borrowing exceeds the market capacity. With only 690,417 available against 795,191 of demand, BTCB migration entries revert after about 690,417 of debt is migrated, a shortfall of 104,774. Because batchForceMigrate is atomic, that revert rolls back the entire batch it occurs in. If not, until more lisUSD is reallocated to accommodate the BTCB/lisUSD market migration, full migration cannot be completed.
Recommendation	Consider sizing the BTCB reallocation to the actual whitelisted BTCB debt rather than 94,000. To cover the full measured BTCB CDP debt the market needs roughly 795,191 minus 596,417, which is about 198,774 of fresh supply, so the reallocation should be near 199k rather than 94k.
Status	[Open/Fixed/Acknowledged]

[L01] Forced migration of BNB ilk positions surrenders the protocol slisBNB yield to users in MARKET_SLISBNB	
Contract	PositionMigrator.sol
Severity	Low
Description	**Description** PositionMigrator.batchForceMigrate moves whitelisted CDP positions into Moolah with no opt in from the owner. For an account holding BNB ilk collateral, _migrate and onMoolahFlashLoan release that collateral as slisBNB and supply it into MARKET_SLISBNB. slisBNB is a yield bearing token whose exchange rate against BNB grows over time. The protocol built the YieldAccount specifically to retain that growth, where it records the depositor's BNB principal, lets the owner withdraw and borrow only against that principal, and skims the slisBNB appreciation above principal to the protocol treasury. Routing a migration into a YieldAccount is hardcoded to a single address, YIELD_ACCOUNT_OWNER. **Impact** Every other whitelisted BNB ilk account is instead supplied through slisBnbProviderLending into a plain, self owned Moolah slisBNB position that has no principal baseline and no skim, so the account keeps the full slisBNB appreciation. The contracts state this directly. As such, the protocol actively gives up its own yield claim

	on the roughly 333k lisUSD of remaining BNB ilk CDP debt, with the previously dominant whale position already migrated and zeroed.
Recommendation	Confirm whether handing the slisBNB appreciation to migrated BNB ilk accounts is intended. If the protocol means to retain that yield beyond the single whale, route all forced BNB ilk migrations through a YieldAccount style wrapper that records BNB principal and skims appreciation to the treasury rather than supplying into a plain position, and replace the single hardcoded YIELD_ACCOUNT_OWNER with a mechanism that can cover every such account. If the residual yield is immaterial and the goal is only to wind down the CDP, document that the forgone yield on the roughly 333k lisUSD of BNB ilk debt is an accepted cost, so the asymmetry is a deliberate decision rather than an oversight.
Status	[Open/Fixed/Acknowledged]

[I01] A single failing entry reverts the entire batch	
Contract	PositionMigrator.sol
Severity	Informational
Description	**Description** batchForceMigrate processes all entries in one atomic transaction. If any single entry reverts, every entry in the call is rolled back, including the provider registration. Several realistic conditions cause a single entry to revert • An account whose CDP debt is already 0 because it was migrated earlier or has already performed CDP debt repayment reverts with "no debt to migrate" • An account not on the whitelist reverts with "not whitelisted" • An entry whose debt exceeds the target market liquidity reverts with INSUFFICIENT_LIQUIDITY. **Impact** Because the operational notes describe a staged campaign (CDP repayment capacity of 1.58M against ~2.58M of whitelisted debt forces multiple flap refills), the batch will be run many times, and one stale or already migrated address in any batch wastes the whole call.
Recommendation	Consider keeping batches small and build each batch from a fresh pre flight read of on chain state.
Status	[Open/Fixed/Acknowledged]

[I02] Re-enabling the migration deadline silently re-arms forced migration against stale whitelisted accounts	
Contract	PositionMigrator.sol

Severity	Informational
Description	**Description** The PositionMigrator contract lets an admin reopen forced migration simply by setting a new deadline through setMigrationDeadline, which re-exposes every account still present on the whitelist to forced migration. The contract never records a permanent flag marking an account as "done." Instead, an account is treated as finished only because its CDP debt happens to be 0 at the moment of migration. If such an account later re-engages with the CDP and opens a fresh position, re-enabling the deadline makes that new position eligible for forced migration again, with no fresh consent from the account owner. **Impact** A re-enable can set the deadline to now + 1, leaving essentially zero voluntary window. Once the TimeLock action lands, re-arming can be effectively instant. The re-enable paths are also asymmetric in reach. Re-enabling the deadline by itself re-allows a single forceMigrate against any whitelisted account that has authorized PositionMigrator on Moolah. Re-enabling the deadline and additionally re-granting the Moolah MANAGER role re-allows the full batchForceMigrate bypass against all whitelisted accounts at once. Together these let the admin reopen non consensual migration against accounts that were previously settled, including positions those users opened fresh after the first campaign, with little or no notice.
Recommendation	Consider ensuring the whitelisted addresses list is up-to-date and does not include previously migrated addresses. If not, consider a separate deadline for the new batch force migration mechanism.
Status	[Open/Fixed/Acknowledged]

[I03] Reallocation should source from the lowest utilization market	
Contract	PositionMigrator.sol
Severity	Informational
Description	**Description** The lisUSD MoolahVault at 0xE03D86e5Baa3509AC4A059A41737bAa8169B6529 funds the migration prerequisite by reallocating roughly 1.11M into the slisBNB target market and roughly 94k into the BTCB target market, about 1.2M in total. The vault holds no idle lisUSD, so that amount must first be withdrawn from its other lisUSD markets. Currently, only two of its markets hold meaningful withdrawable liquidity: the 85% lisUSD/slisBNB market, which is 63.2% utilized with about 11.38M free and has active borrowers, and the 85% lisUSD/WBNB market, which is 0.1% utilized with about 7.22M

	free and is effectively idle. Withdrawing proportionally across both markets would raise the utilization of the active slisBNB market and push up the variable borrow rate paid by its existing borrowers, even though the idle WBNB market can supply the entire 1.2M on its own with almost no effect on anyone. **Impact** Proportional withdrawal therefore spreads an avoidable rate increase, which is less fair to the users who actually transact in the active market, not more. Sourcing from the idle market also preserves the vault's withdrawal buffer in the active market, which matters after the migration drives the slisBNB target market to about 100% utilization and makes that position non withdrawable for vault depositors.
Recommendation	Consider as an operational step, to source the reallocation from the lowest utilization market first rather than withdrawing proportionally. Here the idle 85% lisUSD/WBNB market can cover the full 1.2M while leaving about 6.02M of buffer and disturbing essentially no borrowers, so draw from it before touching the active 85% lisUSD/slisBNB market.
Status	[Open/Fixed/Acknowledged]

[I04] Forced migration repayment depends on global HayJoin capacity refilled by a system wide flap	
Contract	PositionMigrator.sol
Severity	Informational
Description	**Description** Each forced migration repays a CDP position through Interaction.paybackFor, which routes to hayJoin.join. That join can only move internal balance the HayJoin already holds, so a repayment succeeds only while vat.hay(hayJoin) is at least the amount being repaid. The current capacity is 1.58M while the whitelisted debt totals about 2.58M, that is 1.58M < 2.58M, so without a refill any batch, or sequence of batches, whose cumulative repayment exceeds the live capacity reverts inside hayJoin.join. **Impact** The operational prerequisite that lifts capacity has protocol wide effects. • Setting Vow.hump to 0 is a global parameter change that removes the surplus buffer for the entire protocol during the migration window, so any bad debt event, meaning an increase in sin, that occurs in that window has no cushion before surplus is drained, and hump must be restored afterward. • The flap that refills the HayJoin moves the Vow surplus into the HayJoin and mints up to about 3.82M of new circulating lisUSD to the multisig, a one time supply expansion that realizes surplus across the whole stablecoin and is independent of the migration. flap() is permissionless, being external with only

	a surplus check, but it always pays the fixed multisig address, so it is not a theft vector. Any caller simply forwards surplus to the multisig and tops up the HayJoin. Two sequencing hazards follow: 1. First, the refill must happen before the batches, and the hump = 0 flap must precede any batch that repays > 1.58M, or that batch reverts. 2. Second, the HayJoin capacity is a global shared resource and paybackFor is permissionless, so any unrelated CDP repayment that lands between the flap and the migration batches also drains vat.hay(hayJoin) and can starve the migration, reverting a batch. The refill and the migration therefore need to run in tight succession.
Recommendation	Treat the HayJoin capacity as a precondition that is measured and reserved rather than assumed. Before each batch, read vat.hay(hayJoin) and size the batch so its total repayment stays within the available capacity, and run the hump = 0 flap refill immediately before the batches, ideally bundling the refill and the migration into the same block so unrelated repayments cannot drain the capacity in between. Restore Vow.hump to its prior value as soon as the migration completes so the protocol regains its surplus buffer, and account for the roughly 3.82M of lisUSD minted to the multisig by the flap as an expected one time supply change. For resilience, consider having the migrator read the live capacity and skip or defer entries that would exceed it instead of reverting the whole batch.
Status	[Open/Fixed/Acknowledged]